

Characterizing and Optimizing Realistic Workloads on a Commercial Compute-in-SRAM Device

Niansong Zhang
Cornell University
Ithaca, NY, USA
nz264@cornell.edu

Wenbo Zhu*
University of Southern
California
Los Angeles, CA, USA
wenbozhu@usc.edu

Courtney Golden
MIT
Cambridge, MA, USA
cgolden@csail.mit.edu

Dan Ilan
GSI Technology Inc.
Tel Aviv, Israel
dilan@gsitechnology.com

Hongzheng Chen
Cornell University
Ithaca, NY, USA
hzchen@cs.cornell.edu

Christopher Batten
Cornell University
Ithaca, NY, USA
cbatten@cornell.edu

Zhiru Zhang
Cornell University
Ithaca, NY, USA
zhiruz@cornell.edu

Abstract

Compute-in-SRAM 架构为在各类数据密集型应用中实现更高性能和更高能效提供了一种有前景的方法。然而，现有评估在很大程度上依赖于模拟器或小规模原型，这限制了对其在真实环境中潜力的理解。在本文中，我们在真实工作负载下，对一种商用的 Compute-in-SRAM 设备——GSI APU——进行了全面的性能与能耗表征。我们将 GSI APU 与包括 CPU 和 GPU 在内的成熟体系结构进行比较，以量化其能效与性能潜力。我们提出了一个面向通用 Compute-in-SRAM 设备的分析框架，通过对性能权衡进行建模，揭示了基本的优化原则，从而指导程序优化。利用紧密

集成的存算架构所提供的细粒度并行性，需要进行精细的数据管理。为此，我们提出了三种优化方法：通信感知的归约映射、合并 DMA，以及面向广播友好的数据布局。当这些优化应用于大规模语料（10GB–200GB）上的检索增强生成（RAG）任务时，我们的 Compute-in-SRAM 系统能够将检索阶段加速 5.4×–7.5×，并将端到端的 RAG 延迟提升 1.1×–1.8×。系统中的共享片外内存带宽通过模拟 HBM 进行建模，而其他所有组件均在真实的 Compute-in-SRAM 设备上测量。关键在于，该系统在 RAG 任务上的性能可与 NVIDIA A6000 GPU 相匹配，同时具有显著更高的能效（降低 54.4×–117.9×）。这些结果验证了 Compute-in-SRAM 在复杂真实应用中的可行性，并为该技术的发展提供了指导。

*This work was done during an internship at Cornell University.

CCS Concepts

• Hardware → Emerging architectures; • General and reference → Evaluation; • Computing methodologies → Modeling methodologies.

Keywords

Compute-in-SRAM, analytical modeling, energy efficiency, retrieval-augmented generation (RAG)

ACM Reference Format:

Niansong Zhang, Wenbo Zhu, Courtney Golden, Dan Ilan, Hongzheng Chen, Christopher Batten, and Zhiru Zhang. 2025. Characterizing and Optimizing Realistic Workloads on a Commercial Compute-in-SRAM Device. In 58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25), October 18–22, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3725843.3756132>

1 Introduction

存内计算（Compute-in-Memory, CIM）通过将计算单元集成在存储阵列内部或其附近，以减少内存访问开销，从而有望成为一种高能效的数据密集型应用加速方法。在各种 CIM 方法中，Compute-in-SRAM 因其与标准 CMOS 工艺的兼容性以及实现高内存带宽的潜力而脱颖而出。Compute-in-SRAM 系统的体系结构设计及全栈优化持续吸引着大量研究关注。

近年来已有多种设计被提出：Compute Caches [1] 利用 SRAM 位线将缓存单元重用为向量计算单元，在 32 字节 SIMD CPU 上实现了 1.9× 的加速和 2.4× 的能耗降低；EVE [7] 采用位混合执行机制，使向量操作相较于乱序 CPU 获得近 8× 的加速；CAPE [11] 提供了一种可编程的 CMOS 关联计算引擎，在等面积 CPU 上平均实现 14× 的加速，峰值可达 254×。此外，一些专用加速器（如 iMTransformer [30]、TranCIM [45]、PICMA [48] 和 iMCAT [29]）面向深度神经网络（DNN）和 Transformer 模型，进一步展示了面

向特定领域加速的潜力。

This work is licensed under a Creative Commons Attribution 4.0 International License.

MICRO '25, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1573-0/2025/10

<https://doi.org/10.1145/3725843.3756132>



Table 1: Comparison of GSI APU [22, 44], Intel Xeon 8280, NVIDIA A100, and Graphcore IPU.

	GSI APU	Xeon 8280	NVIDIA A100	Graphcore
Compute Cores	2 million×1 bit	28×2×512 bits	104×4,096 bits	1,216×64 bits
Tech node	28 nm	14 nm	7 nm	7 nm
Compute Speed	500 MHz	2.7 GHz	1.4 GHz	1.6 GHz
Peak Compute	25 TOPS	10 TOPS	75 TOPS	16 TOPS
On-Chip Memory	12MB L1	38.5MB L3	40MB L2	300MB L1
Mem. Bandwidth	26 TB/s	1 TB/s	7 TB/s	16 TB/s
Power	60W TDP	205W TDP	400W TDP	150W TDP

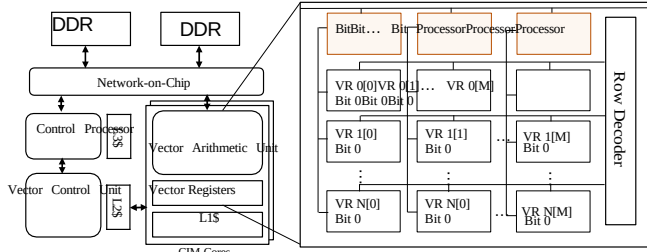


Figure 1: General-purpose compute-in-SRAM system model – CAPE [11], and GSI APU [22, 44] follow the same system abstraction, with different vector arithmetic unit and SRAM cell implementations.

尽管已有令人鼓舞的结果，这些架构主要仍通过指令级建模与仿真 [1, 29, 30, 47, 48] 或小规模原型 [45] 进行评估，这限制了对其在实际真实环境中效果的认识。这一差距凸显了在真实工作负载下对商用 Compute-in-SRAM 设备进行性能表征的必要性，从而弥合理论潜力与实际可行性之间的鸿沟。

通用的 Compute-in-SRAM 系统通常采用 SIMD 向量处理器的抽象模型 [1, 11, 47]。如图 1 所示，一种常见的 Compute-in-SRAM 架构（被 CAPE 等系统采用）将具备计算能力的 SRAM 阵列抽象为一个向量处理引擎。GSI APU [22, 44] 也遵循这一抽象模型，作为一个商用实例，它为在真实工作负载和应用场景下评估 Compute-in-SRAM 系统的潜力提供了独特的机会。

GSI APU 集成了 200 万个位级处理器，运行频率为 500 MHz，在 8 位加法运算下可提供最高 25 TOPS 的算力 [44]。表 1 将其计算能力、内存带宽以及能效与 CPU、GPU 和 ASIC 加速器进行了对比，显示出其在数据密集型工作负载中的强大潜力。然而，要充分发挥这一潜力并不容易：该 APU 采用一个包含 32,768 个元素的向量处理器，其计算与存储按列集成，能够提供 TB/s 级的片上带宽，但限制了向量寄存器（VR）内部的内存访问。例如，跨 VR 的归约操作不被支持，而 VR 内部的分组操作相比 VR 间操作要慢约 10 倍。

图 2 展示了 GSI APU 上不同矩阵乘法内核的屋顶线模型。基线方法采用量化的内积算法实现，但未考虑数据移动或数据布局的开销，因而性能不佳。然而，通过针对性的数据优化，在提高运算强度的同时，性能可以逐渐接近计算上限。

这一观察揭示了关于 Compute-in-SRAM 设备的一个更普遍的认识：尽管计算直接在存储器内部完成，如果数据移动没有得到精

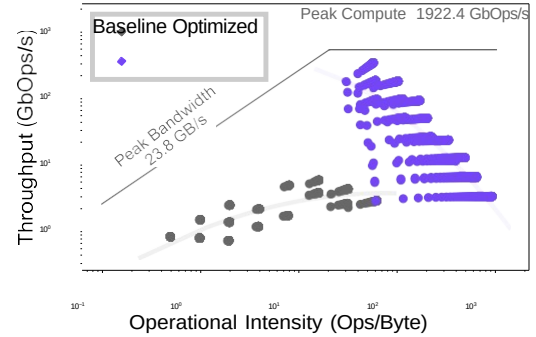


Figure 2: Performance of various matrix multiplication kernels on GSI APU, w/o data movement and data layout optimizations.

细管理，这类系统仍可能受到内存带宽的限制。为进一步分析这一问题，我们构建了一个分析框架，以揭示其潜在的性能上限。在本文中，我们提出了三项关键优化，以充分发挥 Compute-in-SRAM 系统的潜力：通信感知的归约映射、合并的 DMA 操作，以及面向广播友好的数据布局。

¹ 对于 16 位无符号乘法和累加操作，进行了峰值计算边界的分析。

2

此外，我们评估了用于大规模语言模型（LLM）推理的计算内存（compute-in-SRAM）系统在检索增强生成（RAG）任务中的表现，展示了其在该工作负载下的适用性。我们还将性能与能效与 CPU 和 GPU 进行了比较，以突出其优势。我们的贡献如下：

- 我们首次对商业计算内存设备进行了全面评估，采用了现实工作负载。具体来说，我们使用 Phoenix 基准测试、矩阵乘法和检索增强生成（RAG）工作负载，评估了 GSI APU——一种商业化的通用计算内存设备实例，并与 Intel Xeon Gold CPU 和 NVIDIA A6000 GPU 等已建立架构进行了性能和能源效率的比较。
- 我们开发了一个灵活的分析框架，该框架通过调节关键设计参数，识别优化机会并支持架构设计空间探索。这个框架为下一代计算内存架构的设计提供了指导。
- 我们提出了三个关键优化，旨在通过利用超长向量计算内存架构的独特特性，优化数据传输和布局。在 RAG 工作负载中应用这些优化后，与未经优化的基线相比，检索延迟降低了最多 6.5 倍，端到端加速比 CPU 提高了最多 1.78 倍，并且在能耗上减少了最多 99%。在 Phoenix 上，优化后的 APU 相比 CPU 实现了 36.7 倍的加速。

2 GSI APU Architecture

2.1 Architecture and Microarchitecture

在本节中，我们提供了 APU 架构和微架构的概述。如图 3(a)所示，APU 平台由一个标准的 x86-64 主机 CPU 和一个四核 APU 芯片组成，通过 PCIe 连接，并共享一个 DDR4 DRAM。每个 APU 核心作为一个向量引擎，处理 32K 元素的 16 位数据向量，如图 3(b)所示。32 位的 ARC 控制处理器（CP）向向量命令单元（VCU）发出向量命令，VCU 将向量命令解码为微指令操作，以加载/存储向量到向量寄存器（VR）并执行算术计算。

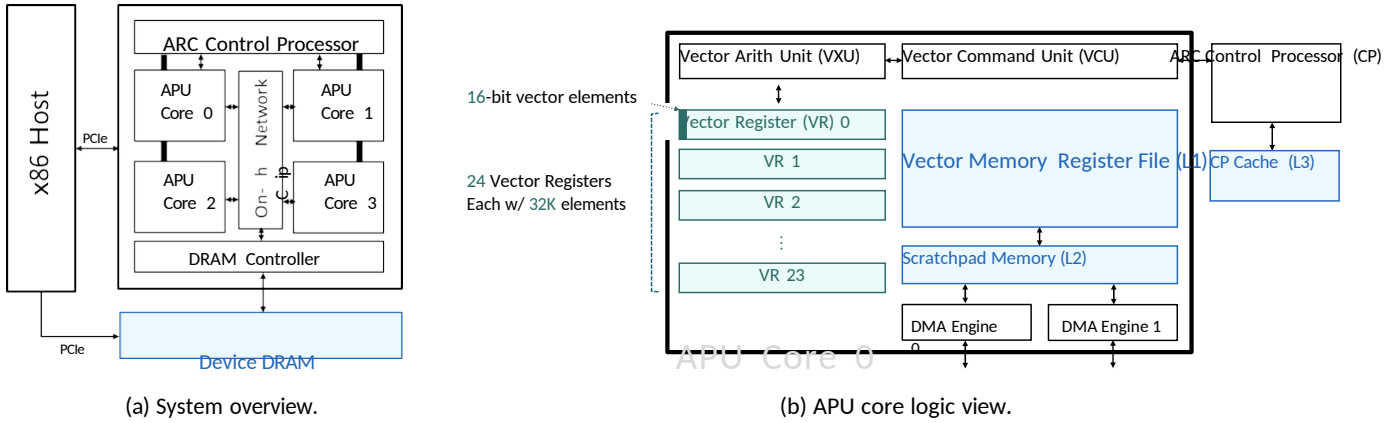


Figure 3: The GSI APU system and APU core logic view, the memory hierarchy is highlighted in blue. The APU consists of a control processor, four APU cores, and a four-level memory hierarchy. Each core has 24 vector registers (VR), and each VR has 32768 elements.

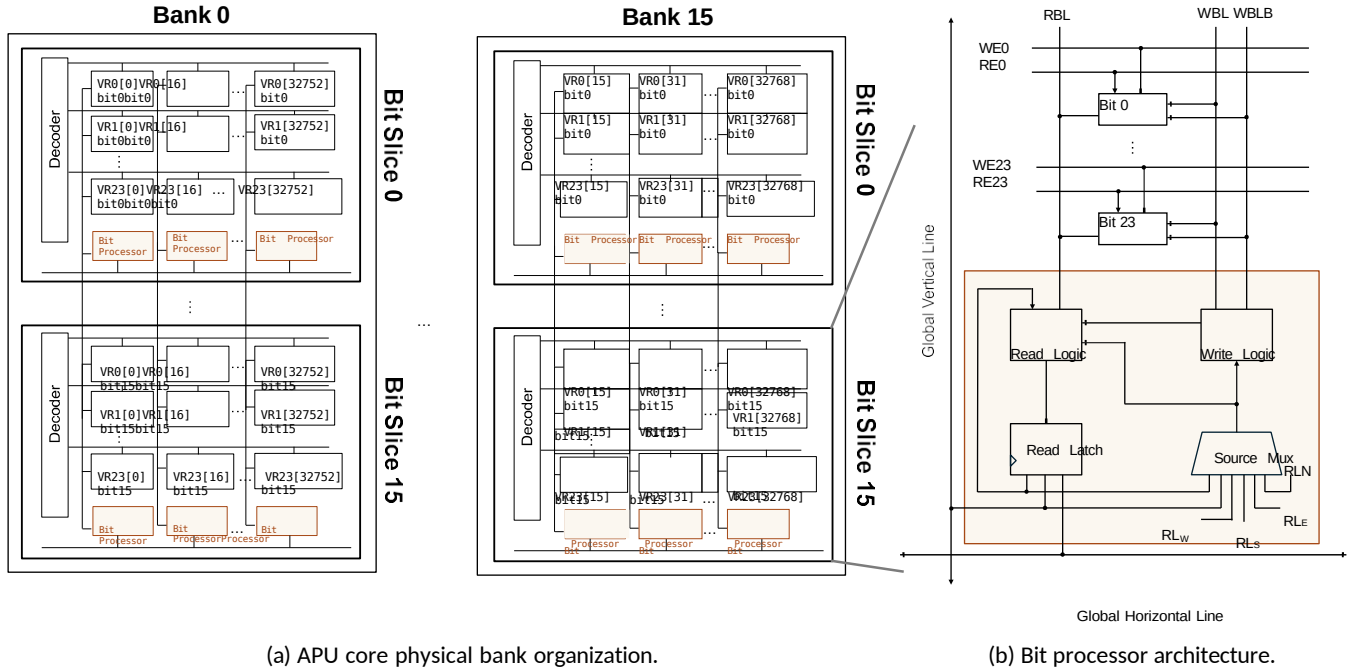


Figure 4: The physical bank organization of one GSI APU core and the bit processor architecture. The data is stored in a bit-slice fashion, each column of each bit-slice contains a single-bit read latch and associated read and write logic.

内存层次结构在图 3 中用蓝色标出, 包含了 16 GB 的设备 DRAM、1 MB 的控制处理器 (CP) 缓存 (L3)、64 KB 的临时存储 (L2) 以及 3 MB 的向量内存寄存器文件 (L1)。DDR 内存由四个 APU 核心共享, 每个核心拥有自己的 L2 和 L1 内存。L2 临时存储作为 DMA 缓冲区, 由于存储一个 32K 元素, 16 位的向量。L1 内存被组织为 48 个“背景”寄存器, 作为计算使能的向量寄存器 (VRs) 的额外存储。

图 4(a)显示了向量寄存器 (VRs) 的物理 bank 组织。24 个 VR 被分布在 16 个物理 bank 中, 每个 bank 包含 2048 个 16 位元素。在一个物理 bank 内, 数据以切片的方式存储, 每个位切片包含所有 24 个 VR 的一个比特。每个位切片的每列集成了一个比特处理器, 包含 24 个定制的 12T SRAM 单元。比特处理器的微架构如图 4(b)所示。比特处理器的集合相当于向量算术单元 (VXU)。读取逻辑可以对两个或更多操作数执行 AND、OR 和 XOR 操作, 包括来自读取比特线 (RBL)、读取锁存器 (RL)、全局垂直线、全局水平线

以及来自其RRRRRR、RRRRRR、RRRRRR或RRRRRR比特处理器的 RL 的数据。

Table 2: Microarchitectural state and operations on state.

State	Description
RL	read latch
GVL	global vertical latch
GHL	global horizontal latch
VR[<i>ii</i>]	vector register source <i>ii</i>
Operations	Description
RL = VR[<i>vvvv ss</i> 0]	read VR
RL = VR[<i>vvvvss</i> 0, <i>vvvvss</i> 1]	read and bitwise AND of two VRs
RL = <i>RR</i>	read value from a source latch
RL = VR[<i>vvvv ss</i> 0] <i>oooo RR</i>	operate on a VR and a latch
RL <i>oooo</i> = VR[<i>vvvv ss</i> 0]	operate on current RL and a VR
RL <i>oooo</i> = <i>RR</i>	operate on current RL and a latch
RL <i>oooo</i> = VR[<i>vvvv ss</i> 0] <i>oooo RR</i>	operate on RL, a VR, and a latch
latch VR[<i>vvvvss</i> 0] = <i>RR</i>	write to VR from source latch

全局水平线连接同一行的所有比特处理器，而全局垂直线连接同一列的比特处理器。每条线包括一个 1 位锁存器：全局水平锁存器（GHL）和全局垂直锁存器（GVL）。如果多个值同时读取到 GHL，则在将结果存储到锁存器之前执行 OR 操作。对于 GVL，则对多个值执行 AND 操作。写入逻辑通过写入比特线（WBL）或其反向（WBLB）更新 SRAM 单元。默认情况下，所有位切片中的比特处理器会发出相同的微操作。然而，可以使用 16 位掩码对部分位切片执行操作。VRs、RL、GHL 和 GVL 是微架构状态。微架构状态上的操作列在表 2 中。

2.1.1 算术操作和数据类型。与一次仅处理一个比特的位序列架构不同，APU 支持位序列算术操作和位并行布尔操作。这种灵活性是通过图 4(a)所示的位切片 bank 组织实现的，允许所有位的 VR 可以被比特处理器同时访问。APU 原生支持 16 位有符号和无符号整数、16 位 IEEE 浮点数，以及自定义的 GSI 浮点格式，具有 6 位指数和 9 位尾数。

2.1.2 数据传输。APU 支持直接内存访问（DMA）和可编程输入输出（PIO）。如图 3(b)所示，每个 APU 核心配备了两个并行 DMA 引擎，能够以 512 字节为单位传输连续数据，从而为内存层次结构中的 VR 传输提供高带宽。对于从 VR 中进行随机访问或单元素提取，ARC 控制处理器可以使用 PIO 执行这些操作。对于 DRAM $\leftrightarrow$ L3 的传输，可以使用 DMA 和 PIO，而对于 DRAM $\leftrightarrow$ L2，则仅支持 DMA。

DRAM $\leftrightarrow$ L3, DRAM $\leftrightarrow$ L2: 对于这些类型的数据传输，可以应用数据布局转换。使用 DMA 时，源和目标 512 字节块的地址可以进行编程，以启用连续、跨步和重复的数据布局转换。PIO 虽然支持任意的数据布局转换，但与 DMA 相比带宽较低。

L2 $\leftrightarrow$ L1, L1 $\leftrightarrow$ VR: 对于这些类型的数据传输，不支持数据布局转换。数据是以整个向量的粒度进行传输，意味着只能进行完整的 VR 加载/存储（32K $\times$ 16 位）。L3 $\leftrightarrow$ VR: PIO 支持通过响应 FIFO（RSP FIFO）实现 L3 和 VR 之间的直接数据传输。它支持从 VR 的串行检索（get）和向 VR 的并行插入（set）。控制处理器（CP）可以将标量或立即数广播到整个 VR 或被掩码的元素，而从 VR 检索数据时仅限于一次检索一个元素。

Inter-VR vs. intra-VR: 由于位切片组织，VR 之间的逐元素数据传输可以高效执行，因为所有元素和位可以由比特处理器并行处理。然而，VR 内部的数据传输，如向量位移或 bank 复制，依赖于 GHL 或 RSP FIFO，因此无法完全并行化。

2.1.3 不同的 VR 间和 VR 内数据传输成本影响了内存层次结构中数据布局对性能的多方面影响：

对于设备 DRAM 和 L3，数据布局影响数据传输的带宽。当数据是连续的或具有规则的跨步时，DMA 提供的带宽高于 PIO。VR 内的数据布局也影响数据传输时间。如果计算结果在 VR 内是连续的，DMA 可以高效地将它们传输回 L1、L2，然后到设备 DRAM。然而，如果结果是分散的，则必须使用 PIO 按顺序移动它们。VR 内的数据布局影响计算效率。例如，归约操作可以映射到 VR 间操作或 VR 内操作，取决于 VR 中的数据布局。如第 3.2 节所讨论，VR 内的数据传输比 VR 间传输更具成本，因此由于数据传输开销，VR 内的归约操作更为昂贵。

2.2 Programming Model

APU 采用主机-加速器编程模型，其中 x86 主机管理 APU 设备上的内核执行、共享内存和内核调用。图 5 使用一个简单的向量加法示例来演示主机与设备之间的交互。

2.2.1 主机程序。主机程序使用 C 语言编写，管理内核调用、共享 DRAM（L4）内存的分配和回收，以及主机和设备内存之间的数据传输。图 5(a)显示了主机端代码的一部分。为了简洁起见，省略了调用上下文和输入数据的初始化。主机和设备通过程序命令结构进行通信，具体在 L1-L9 行中进行了详细说明。内存管理，包括设备内存分配、数据传输和内核调用，由 GSI GDL 库处理，这是 GSI 提供的内存管理库。

2.2.2 Device Program.设备程序（Device Program）同样使用 C 语言编写，在 APU 控制处理器上运行，并采用通用控制流语句。系统宏 GAL_TASK_ENTRY_POINT 定义了设备程序的入口点，负责从命令中提取数据结构并调用 vec_add 函数。设备程序管理从设备内存到 L1 内存的数据传输，并使用向量寄存器

（VRs）执行向量计算。向量处理依赖 GSI 向量数学库

(GVML)，该库提供包括算术、逻辑、按位、三角函数及最值运算在内的向量操作函数。一旦计算完成，设备程序将数据传回设备内存。

```

1 // define APU program data and command
2 struct program_data {
3     uint64_t mem_hndl_vec1, mem_hndl_vec2, mem_hndl_out;
4 } __attribute__((packed));
5
6 struct program_cmd {
7     char* program_name;
8     struct program_data data;
9 } __attribute__((packed));
10
11 const uint64_t vec1_size = sizeof(uint16_t) * LENGTH;
12 const uint64_t vec2_size = sizeof(uint16_t) * LENGTH;
13 const uint64_t out_size = sizeof(uint16_t) * LENGTH;
14 const uint64_t total_io_size = vec1_size + vec2_size + out_size;
15
16 // Allocate device DRAM memory
17 gdl_mem_handle_t L4_buf = gdl_mem_alloc_aligned(total_io_size);
18 // Add device DRAM ptrs to the program command
19 struct program_cmd cmd = {
20     .data.mem_hndl_vec1 = L4_buf,
21     .data.mem_hndl_vec2 = L4_buf + vec1_size,
22     .data.mem_hndl_out = L4_buf + vec1_size + vec2_size
23 };
24 // Copy data from host to device DRAM
25 gdl_mem_cpy_to_dev(&cmd.data.mem_hndl_vec1, vec1_host, vec1_size);
26 gdl_mem_cpy_to_dev(&cmd.data.mem_hndl_vec2, vec2_host, vec2_size);
27 // Copy program cmd to device DRAM
28 gdl_mem_handle_t L4_cmd = gdl_mem_alloc_aligned(sizeof(cmd));
29 gdl_mem_cpy_to_dev(L4_cmd, cmd, sizeof(cmd));
30 (a) APU Host code.
31
32 static int vec_add(struct program_data *data) {
33     // convert mem handles to device DRAM pointers
34     void *vec1_L4ptr =
35         gal_mem_handle_to_apu_ptr(data->mem_hndl_vec1);
36     uint16_t *vec2_L4ptr =
37         (uint16_t *)gal_mem_handle_to_apu_ptr(
38             data->mem_hndl_vec2);
39     uint16_t *out_L4ptr =
40         (uint16_t *)gal_mem_handle_to_apu_ptr(
41             data->mem_hndl_out);
42     // move data from device DRAM to L1
43     direct_dma_l4_to_l1_32k(GVML_VM_0, vec1_L4ptr);
44     direct_dma_l4_to_l1_32k(GVML_VM_1, vec2_L4ptr);
45     // load to VR, perform addition, store to L1
46     gvm1_load_32(vec1_vr, GVML_VM_0);
47     gvm1_load_32(vec2_vr, GVML_VM_1);
48     gvm1_add_u16(result_vr, vec1_vr, vec2_vr);
49     gvm1_store_16(GVML_VM_3, result_vr);
50     // move data from L1 to device DRAM
51     direct_dma_l1_to_l4_32k(out_L4ptr, GVML_VM_3);
52     return 0;
53 }
54
55 GAL_TASK_ENTRY_POINT(vec_add_task, in) {
56     struct program_cmd *cmd = (struct program_cmd *)in;
57     gvm1_init_once();
58     return vec_add(&cmd->data);
59 }
60 (b) APU device code.

```

Figure 5: Simple vector addition code demonstrating APU programming model.

GVML 库是通过 APU 微码指令实现的。APU 微码指令可直接操作表 2 所列的微架构状态。APU 程序员也可以使用微码指令实现不同的向量抽象。例如，Golden 等人 [19] 使用 APU 微码实现了 RISC-V 向量抽象。在本工作中，我们使用 GVML 提供的抽象，以便专注于通过这些数据移动和计算操作进行性能优化。

Table 3: Notations

Notation	Description	Notation	Description
dd	Data size in bytes	BW	Memory bandwidth
vr	VR group	ss	Subgroup size
$\sigma\sigma$	size Lookup table size	CC	Constant

3 Analytical Framework

我们提出了一种灵活的分析框架，用于建模计算-内存（compute-in-SRAM）平台的性能特性。该框架对关键架构因素进行参数化，包括计算延迟、数据移动带宽以及可能存在非均匀开销的通信模式。这种通用性使框架可应用于多种计算-内存架构，有助于在特定设备实现之外进行性能分析和优化策略制定。

3.1 Applicability and Assumptions

该分析框架针对如图 1 所示的计算-内存（compute-in-SRAM）系统模型。模型假设采用基于 PCIe 的加速器，具有多级存储层次和向量处理器抽象，其中数据移动成本在不同存储层级以及向量寄存器内部存在非均匀性。虽然该框架使用 GSI APU 进行验证，但并不限于该设备。通过性能分析得到必要参数，该框架可推广到遵循相同系统模型的其他计算-内存平台。表 3 总结了框架中使用的符号表

示。表 4 和表 5 分别提供了数据移动和计算操作的通用延迟模型。第 5.2.2 节讨论了框架针对真实设备测量延迟的验证情况。

3.2 Data Movement

在计算-内存（compute-in-SRAM）系统中，高效的数据移动对于数据密集型应用尤为关键。以下讨论了这些架构通常支持的主要数据移动范式。

3.2.1 DMA Transfers. 直接存储器访问（Direct Memory Access, DMA）操作可以在计算-内存平台上高效地完成大块数据传输。DMA 的延迟通常随传输大小线性增长，其模型为： $T_{DMA} = dd/BW + T_{init}$ ，其中， dd 为数据大小， BW 为带宽， T_{init} 为固定的初始化开销。虽然 DMA 对连续数据传输具有高吞吐量，但对于非常大的数据量，片外存储器带宽限制可能成为性能瓶颈。

3.2.2 Programmable I/O (PIO). PIO 支持细粒度的、非规则的数据访问模式。PIO 传输的延迟通常随单个加载或存储操作的数量线性增长，其模型为：

Table 4: Data movement analytical framework

Operation	Description	Execution Time (cycles)	
		Analytical	Meas.
dma_l4_l3	L4→L3 DMA	$dd/BW + TT_{init}$	$0.19dd + 41164$
dma_l4_l2	L4→L2 DMA	$dd/BW + TT_{init}$	$0.63dd + 548$
dma_l2_l1	L2→L1 DMA, 16-bit × 32K	$TT_{l2→l1}$	386
dma_l4_l1	L4→L1 DMA, 16-bit × 32K	$TT_{l4→l1}$	22272
dma_l1_l4	L1→L4 DMA, 16-bit × 32K	$TT_{l1→l4}$	22186
pio_ld	PIO load, L4→VR	$ii \cdot TT_{pio_ld}$	$57ii$
pio_st	PIO store, VR→L4	$ii \cdot TT_{pio_st}$	$61ii$
lookup	Lookup L3 w/ index VR	$CC \cdot \sigma\sigma + TT_{init}$	$7.15\sigma\sigma + 629$
load_store	VR ↔ L1 load store	TT_{ld}, TT_{st}	29
cpy	VR ↔ VR elementwise copy	TT_{cpy}	29
cpy_subgrp	Copy VR subgroup to group	TT_{cpy_sgp}	82
cpy_imm	Broadcast an immediate to VR	TT_{cpy_imm}	13
shift_e(k)	Shift VR entries to head/tail by kk	$CC \cdot kk$	$373kk$
shift_e(4k)	Shift VR entries to head/tail by $4k$	$CC \cdot 4k$	$88 \cdot 4k$

* In the analytical framework, we refer to the device DRAM as L4 memory.

$TT_{PIO} = ii \cdot TT_{access}$, 其中, ii 为操作次数。尽管灵活, PIO 相较于 DMA 开销更高, 因此主要适用于非连续或稀疏数据的传输。

3.2.3 IndexedLookupandElement-WiseOperations. 索引查找用于处理从高层存储器到本地向量寄存器 (VR) 的非规则、类似散点的数据传输。查找延迟通常随查找表大小 $\sigma\sigma$ 增长, 其模型为: $TT_{lookup} = CC \cdot \sigma\sigma + TT_{init}$, 这强调了优化索引和数据布局的重要性。按元素拷贝操作, 如标量广播和 VR 间传输, 由于硬件并行机制支持, 通常以恒定时间完成。这类操作对于数据初始化和并行任务中的广播操作至关重要。

3.2.4 Vector Register (VR) Shifts. VR 内部的移位操作可以在寄存器内局部重排数据, 而无需访问外部存储器。其延迟与移位幅度成正比, 模型为: $TT_{shift_e} = CC \cdot kk$. 通过优化数据布局以最小化 VR 内移位, 可以显著提高整体性能。

3.3 Computation

在基于 SRAM 的计算平台上, 量化的算术、逻辑和比较操作通常由于其固有的并行执行而以恒定时间完成。因此, 我们在表 5 中总结了它们的符号表示, 并给出了从 GSI APU 获得的代表性延迟测量值。

归约 (Reduction) 操作用于聚合向量寄存器中的元素, 例如求和或寻找极值。此类操作通常采用基于子组的分层归约策略以充分利用并行性。然而, 由于硬件限制, 子组间的归约可能具有非线性开销, 并且其成本可能远高于子组内操作。基于子组的归约的通用成本模型可以表示为:

$$\begin{aligned}
 TT_{sg_add}(vv, ss) &= 003(\log_2 ss)^3 + 002(\log_2 ss)^2 + 001 \log_2 ss + 000, \\
 003 &= \alpha\alpha_3 \cdot \log_2 vv + \beta\beta_3, \quad 002 = \alpha\alpha_2 \cdot \log_2 vv + \beta\beta_2, \\
 001 &= \alpha\alpha_1 \cdot \log_2 vv + \beta\beta_1, \quad 000 = \alpha\alpha_0 \cdot \log_2 vv + \beta\beta_0.
 \end{aligned} \quad (1)$$

Table 5: Computation analytical framework

Operation	Description	Execution Time (cycles)	
		Analytical	Meas.
and_16 or_16	16-bit bit-wise and	TT_{and}	12
not_16 xor_16	16-bit bit-wise or	TT_{or}	8
ashift	16-bit bit-wise not	TT_{not}	10
add_u16	16-bit bit-wise xor	TT_{xor}	12
add_s16	int16 arithmetic shift	TT_{ash}	15
sub_u16	uint16 element-wise addition	TT_{uadd}	12
sub_s16	int16 element-wise addition	TT_{sadd}	13
popcnt_16	uint16 element-wise subtraction	TT_{sub}	15
mul_u16	int16 element-wise subtraction	TT_{ssub}	16
mul_s16	16-bit population count	TT_{popcnt}	23
mul_f16	uint16 element-wise multiplication	TT_{umul}	115
div_u16	int16 element-wise multiplication	TT_{smul}	201
div_s16 eq_16	float16 element-wise multiplication	TT_{fmul}	77
gt_u16 lt_u16	uint16 element-wise division	TT_{udiv}	664
lit_gf166	int16 element-wise division	TT_{sdiv}	739
ge_u16 le_u16	16-bit element-wise equal	TT_{eq}	13
recip_u16	uint16 element-wise greater than	TT_{ugt}	13
exp_f16	uint16 element-wise less than	TT_{ult}	13
sin_fx cos_fx	gsi float16 element-wise less than	TT_{flt}	45
count_m	uint16 greater than or equal	TT_{uge}	13
add_subgrp_s16	uint16 less than or equal	TT_{ule}	13
	uint16 element-wise reciprocal	TT_{recip}	735
	float16 exponential	TT_{exp}	40295
	fixed-point sine	TT_{sin}	761
	fixed-point cosine	TT_{cos}	761
	count marked entries	TT_{cnt_m}	239
	int16 add sub groups in each group	Eq. 1	-

```

1 framework = LatencyEstimator() # Initialize analytical framework
2 with framework.ctx():
3     total_data_size = 1024 * 1024 * 256 * 3
4     tile_data_size = 8 * 1024 * 48 # Size of one tile across 48 VMRS
5     tile_num = int(total_data_size / tile_data_size)
6     for tile in range(tile_num):
7         for vmr in range(48):
8             for t in range(2):
9                 fast_dma_l4_to_l2(32 * 512) # L4 to L2 DMA
10                direct_dma_l2_to_l1_32k() # L2 to L1 DMA
11
12    for vmr in range(48):
13        gvm_l_load_16()
14        for subgrp in range(8):
15            gvm_l_cpy_subgrp_16_grp(8192, 1024)
16            gvm_l_create_grp_index_u16()
17            gvm_l_cpy_imm_16()
18            for hist_grp in range(8):
19                gvm_l_cpy_16_msk() # Masked copy
20                gvm_l_sr_imm_16() # Shift right by immediate
21                gvm_l_eq_16()
22                gvm_l_cpy_from_mrk_16_msk()
23                ...
24
25    for res_vr in range(8):
26        gvm_l_store_16()
27        direct_dma_l1_to_l4_32k()
28    latency = framework.report_latency() # Estimate total latency
29    print(f"Latency: {latency} us")

```

Figure 6: An example of modeling application latency with the analytical framework. We developed a Python function library with an interface similar to the GSI-provided C++ API. This example models the latency of the **Histogram** application from the Phoenix benchmark suite [41].

三次项的出现源于分层归约 (hierarchical reductions) 中固有的多级移位、对齐和累加操作, 其复杂度随着子组 (subgroup) 大小的增加呈非线性增长。在模型中使用对数 ($\log_2 ss$ and $\log_2 vv$) 是自然的, 因为这些操作通常以阶段方式组织数据聚合, 每一步将子组大小减半, 呈现对数关系。多项式系数 $00_0, 00_1, 00_2, 00_3$ 本身依赖于组大小 vv 的对数, 通过实验确定的常数 $\alpha\alpha_i, \beta\beta_i$ 参数化。这一广义公式可以建模分层归约操作中常见的复杂非线性硬件行为。

3.4 Framework Implementation

我们开发了一个 Python 函数库，其接口与 GSI 提供的 C++ API 保持一致，使程序员能够建模任意 APU 程序。分析框架解释这些程序并报告总延迟。图 6 展示了一个建模 Phoenix 基准套件 [41] 中 Histogram 应用的代码片段。

3.5 Framework Implications

我们的分析框架揭示了计算 SRAM 架构中的整体性能趋势。具体来说，按元素 (element-wise) 计算延迟低，能够高效利用并行硬件。相反，大规模归约操作和某些 VR 内数据移动可能成为显著瓶颈。DMA 传输在批量移动中优于 PIO，但在稀疏数据访问中灵活性不足。因此，在计算

SRAM 平台上实现高效性能需要仔细优化数据移动策略、数据布局 and 计算结构，使其与底层硬件特性对齐。

4 Optimizing Realistic Workloads on Compute-in-SRAM

计算 SRAM 在数据并行性和能效上具有显著优势，但也带来了特定的优化挑战 and 机会。这里，我们以二值矩阵乘法为示例，说明三种关键优化方法。

4.1 Motivating Example

(6)

二值矩阵乘法是机器学习中的重要内核，支持二值神经网络 [50,51] 和二值化 Transformer [31,46] 等工作负载。计算 SRAM 平台在逻辑操作和整数加法方面高效快速，因此非常适合该内核。然而，没有仔细考虑数据布局和数据移动，很难达到高性能。

在图 7 的示例中，输入矩阵 A (MM, MM) 和 B (MM, RR) 沿 MM 轴打包为 uint16 类型。二值矩阵乘法生成输出矩阵 C (MM, RR)，类型为 int16。算法如图 7(a) 所示。要在超长向量处理器上实现该内积算法，基线方法展开循环 jj ，形成图 7(c) 所示的数据布局。我们称这种循环映射方案为空间归约向量映射 (spatial reduction vector mapping)，因为归约求和在 VR 内按空间发生。设 VR 长度为 $ll=32768$ 。假设设备 DRAM 为片外内存，矩阵 B 可放入 L1，则基线操作强度 (OI) 为：

$$OOOO = \frac{MM \cdot RR \cdot (MMMM \cdot \lfloor ll / MMMM \rfloor + \alpha \cdot MM \cdot RR + MMRR) \cdot sf(ll)}{MM}, \quad (2)$$

其中 $\lfloor ll/MM \rfloor$ 是循环 jj 展开的矩阵 A 复制因子， α 为每个标量的逻辑和算术操作数， $sf()$ 表示 $size_of()$ 。矩阵 A 行在 DRAM→L2 中复制，并移动到 L1，其运行时开销为：

$$TT_{AA} = \frac{MM \cdot sf(ll)}{BW} + TT_{init} \cdot \frac{ll}{MM} \cdot \frac{MM}{MM} + MM \cdot TT_{l2 \rightarrow l1}. \quad (3)$$

假设矩阵 B 在设备 DRAM 中按列优先存储，并可放入 L1，则矩阵 B 的移动成本为：

$$TT_{BB} = \frac{RR}{BW} \cdot TT_{l4 \rightarrow l1}. \quad (4)$$

对于 VR C 中非连续结果，PIO 将每个元素传输到 L4，成本为：

$$TT_{CC} = MM \cdot RR \cdot TT_{pio_st} \quad (5)$$

计算运行时成本为：

$$TT_{MAC} = \frac{R \cdot R}{\lfloor ll/MM \rfloor} \cdot (TT_{xor} + TT_{popcnt} + TT_{ash} + TT_{ssub} + TT_{sg_add} + TT_{(MM,1)}) \quad (6)$$

总运行时成本为数据移动成本和计算成本之和。

4.2 Communication-Aware Reduction Mapping

如分析框架所示：(1) VR 内操作成本高于 VR 间操作；(2) 使用 DMA 传输相同数据比 PIO 便宜得多。基于此，我们将二值矩阵乘法实现为标量-向量乘 (SVP) [13]。如图 8 所示，归约轴映射到每次 kk 循环迭代中更高效的按元素操作。我们称这种循环映射为时间归约向量映射 (temporal reduction vector mapping)。同时，输出数据布局变为连续，使快速 DMA 成为可能。因此，计算运行时成本和矩阵 C 移动成本降为：

$$TT_{MAC} = (TT_{xor} + TT_{popcnt} + TT_{ash} + TT_{ssub} + TT_{sadd}) \cdot \frac{MM}{\lfloor ll/RR \rfloor} \cdot MM, \quad (7)$$

7

$$TT_{CC} = \frac{MM}{ll/RR} \cdot TT_{l4 \rightarrow l1}. \quad (8)$$

由于所有位处理器并行操作，更高的 VR 占用率意味着更高的计算效率。为此，我们展开循环 ii 以充分利用 VR，如图 9(a) 所示。循环 ii 的展开保持了循环 kk 的时间映射。结果，VR 布局中出现两级数据复制，如图 9(b) 所示：矩阵 A 的标量因循环 jj 空间展开而复制，矩阵 B 的行因循环 ii 空间展开而复制。这一数据布局创造了数据复用和内存访问合并的机会。我们将矩阵 A 的标量复制实现为来自 L3 的查找操作。因此，标量-向量乘的 OI 为

```

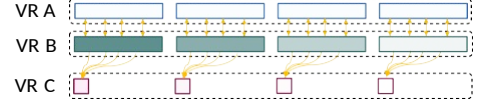
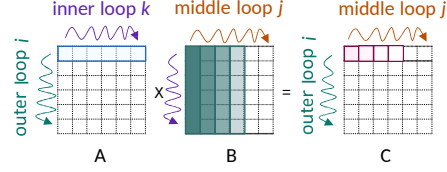
A = uint16 [M,
K] B = uint16
[K, N] C =
int16 [M, N]

```

```

for i in range(M):
    for j in range(N):
        for k in range(K):
            v0 = A[i, k] ^ B[k,
j] v1 = popcount(v0)
            C[i, j] += 16 - v1 <<
1

```

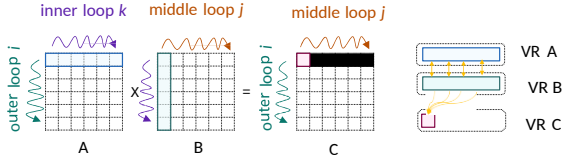


(a) Binary matrix multiply.

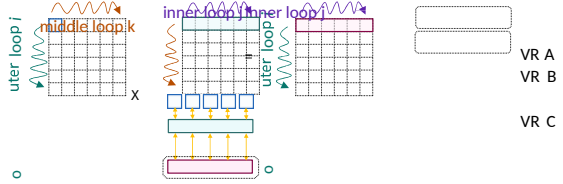
(b) Inner-product data access.

(c) Data layout on vector registers.

Figure 7: Motivating example: binary matrix multiply implemented as an inner-product algorithm on APU.

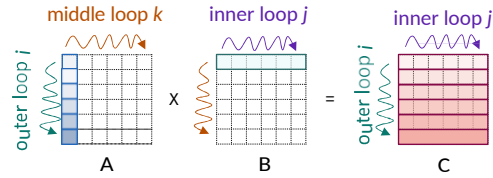


(a) Inner-product maps reduction to intra-VR operations (spatial)

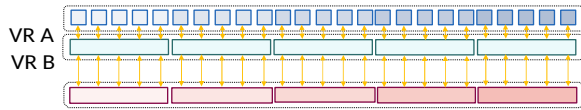


(b) Scalar-vector product maps reduction to inter-VR operations (temporal)

Figure 8: Reduction axis spatial vs. temporal mapping.



(a) Spatially unroll loop i.

VR
C

(b) VR data layout for SVP with spatially unrolled loop i.

Figure 9: Spatially unrolling ii -axis fully utilizes the VR, enables inter-VR reduction, and achieves a contiguous layout of results.

$$TT_{BB} = \frac{MM \cdot RR \cdot MM \cdot \alpha}{(MMMM + RR \cdot MM \cdot \lceil U/RR \rceil + MMRR) \cdot sf(u16)} \quad (9)$$

4.3 DMA Coalescing

在优化数据布局后，新的瓶颈出现在数据复制上。如图 9 所示，一种数据复制形式是将数据块复制到整个向量寄存器 (VR) 中。在图 10(a) 中，我们可以看到 DMA 事务可用于数据复制。然而，这种方法带宽效率低，因为访问片外内存延迟较高，多次 DMA 事务还会增加启动开销。由于

来自矩阵 BB 的相同数据块在循环 kk 的每次迭代中都会被访问，我们可以将这些 DMA 访问合并，以避免冗余数据移动。具体来说，我们将矩阵 BB 的多行 DMA 事务合并为一次事务，从而最小化启动开销。

为了实现这一点，我们引入了一个复用 VR (reuse VR) 来存储初始 DMA 结果。利用子组复制 (subgroup copy) 功能，每行矩阵 BB 被安排在一个子组中，并在循环 kk 的每次迭代中复制以填充整个 VR。值得注意的是，子组复制还可以针对 VR 的一部分进行操作，在仅需复制部分数据时提供了灵活性。这一优化降低了移动矩阵 BB 的运行开销。

$$TT_{BB} = \frac{MM \cdot RR}{TT} \cdot \lceil U/RR \rceil + MM \cdot TT_{cpy_sgp} \quad (12)$$

Since DMA coalescing also removes duplicate data movement from L4, the OI is also improved:

$$OOOO = \frac{MM \cdot RR \cdot MM \cdot \alpha}{(MMMM + RR \cdot MM + MMRR) \cdot sf(u16)} \quad (13)$$

4.4 Broadcast-Friendly Data Layout

$$TT_{AA} = MM \cdot MM/BW + TT_{init} + TT_{lookup}(RR \cdot \lceil U/RR \rceil \cdot MM) \quad (10)$$

在去除冗余的 DMA 操作后，瓶颈转移到用于广播矩阵 A 中标量的查找操作上。如表 4 所示，查找延迟随着查找表大小增加而上升，因此我们需要减小查找表的大小。图 11 展示了查找操作，其中每次广播三个标量，用蓝色填充框标出。在图 11(a) 所示的行优先 (row-major) 布局中，广播窗口最初覆盖索引 0、6 和 12，然后在下一次迭代中移动到索引 1、7 和 13。由于查找表必须是内存中的连续块，为了广播前三行，查找表的大小至少为 18。

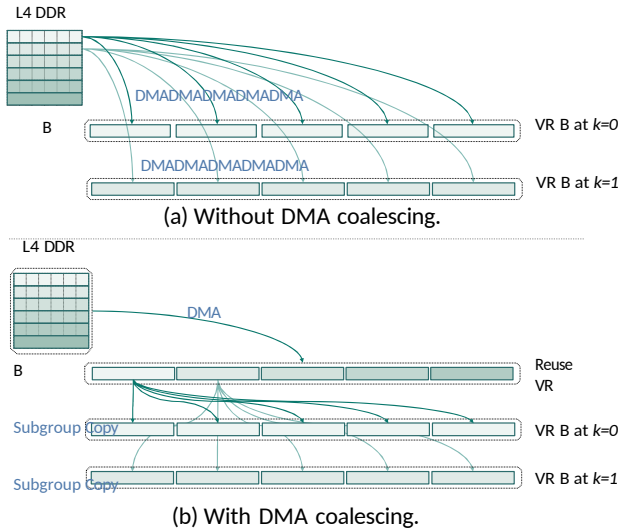


Figure 10: Coalescing DMA - leverage subgroup copy to remove redundant data access and increase data reuse.

为了减少查找表大小，我们将数据布局改为广播友好格式，如图 11(b) 所示。广播窗口最初覆盖索引 0、1、2，然后移动到 3、4、5。因此，查找表大小可减少至 3。我们将这种数据布局表示为维度大小和步长 (strides)，其中分解后的大小和步长用元组表示，如图 11 所示。这种格式由 Graphene [23] 提出。在示例中，该优化将矩阵 A 的广播查找表大小从 $MM \cdot RR$ 降低到 RR ，从而降低数据移动成本，公式如下：

$$TT_{AA} = MM \cdot MM / BW + TT_{init} + TT_{lookup}(RR) \cdot \frac{MM}{\lfloor U/RR \rfloor} \cdot MM. \quad (14)$$

总之，我们展示了这三种关键数据布局与移动优化如何降低计算 SRAM 设备的输入/输出传输成本和计算成本。

5 Evaluation

以 GSI APU 作为商业示例，本节验证分析框架并评估应用优化后的 SRAM 内计算在真实工作负载中的性能。首先，通过二值矩阵乘法延迟分解展示每个优化的贡献；接着通过基准测试验证分析框架，并识别适合 SRAM 内计算的工作负载特征；最后，在大规模语料上进行端到端的 RAG 测试，将 SRAM 内计算的性能与 CPU、GPU 平台进行对比，同时评估能效。

我们使用 GSI Leda-E APU (500 MHz 时钟)、Intel Xeon Gold 6230R CPU (2.1 GHz, L1 缓存 1.6 MB, L2 缓存 52 MB, L3 缓存 71.5 MB) 以及 NVIDIA A6000 GPU 进行对比。

GSI APU 的延迟通过控制处理器周期计数获得，能耗分析使用 Texas Instruments UCD9090 电压监测器和 Renesas 电源模块。

板载 ISL8273M 电源模块提供负载点电压调节和电流监测。

5.1 Binary Matrix Multiplication

我们使用 1024×1024 的二值矩阵乘法内核作为微基准，用于分析和展示所提优化的效果。

图 12 展示了从基线实现到优化版本的延迟分解。关键操作包括：LD LHS / RHS：通过 DMA、PIO 或查找表从片外内存加载矩阵到 L1。VR Ops：片上操作，如子组复制和计算。ST：将结果存回片外内存。我们使用内积算法作为基线实现（见 3.2 节），其瓶颈在于结果数据移动，由于非连续输出的 PIO 存储开销较大。应用通信感知归约映射 (opt1) 通过启用高效 DMA 传输降低了此开销，但增加了 RHS 矩阵加载时间，因为需要数据复制。增加 DMA 合并 (opt2) 进一步改进了 LHS 加载，将 PIO 替换为更快的 DMA，但需额外的向量寄存器操作来完成子组复制。引入广播友好数据布局 (opt3) 也加速了 LHS 加载，但总体瓶颈仍在结果写回。当三种优化结合使用时，结果数据变为连续，可以通过 DMA 传输。我们还对 RHS 矩阵应用了 k 轴数据打包的 DMA 合并，并对 LHS 矩阵广播采用广播友好格式 (1, 2048)_32，最终端到端延迟为 12.0ms，相较基线延迟 226.3ms 提升了 18.9×。

该结果表明，虽然单个优化可能只带来适度加速，但它们为进一步优化创造了机会。例如，通信感知归约映射使 RHS 矩阵 DMA 合并成为可能，并促进了 LHS 矩阵使用广播友好布局。最终，三种优化结合带来了显著性能提升。总之，我们展示了这三种关键数据布局与移动优化如何降低计算 SRAM 设备的输入/输出传输成本和计算成本。

5.2 Phoenix Benchmarks

我们使用 Phoenix 基准套件 [41] 评估优化后的 GSI APU 性能，并与两种 CPU 基线（单线程与多线程）进行对比。选择 Phoenix 的原因在于其数据密集型基准和在 SRAM 内计算研究（如 CAPE [11]）中的使用。官方 Phoenix 基准提供优化 CPU 实现（单线程或多线程），多线程版本使用 MapReduce 编程模型，最多支持 16 线程。表 6 总结了基准套件中的八个应用，包括 CPU 指令计数（通过 Valgrind 测量）和 APU μ Code 指令计数（通过向量指令单元报告）。输入规模范围为 10 MB 到 1.5 GB。我们测量 APU 内核总延迟，包括从设备内存到 L1 及返回的整个数据移动过程。通过应用所有提出的数据移动和布局优化，我们实现了这些基准的优化性能。

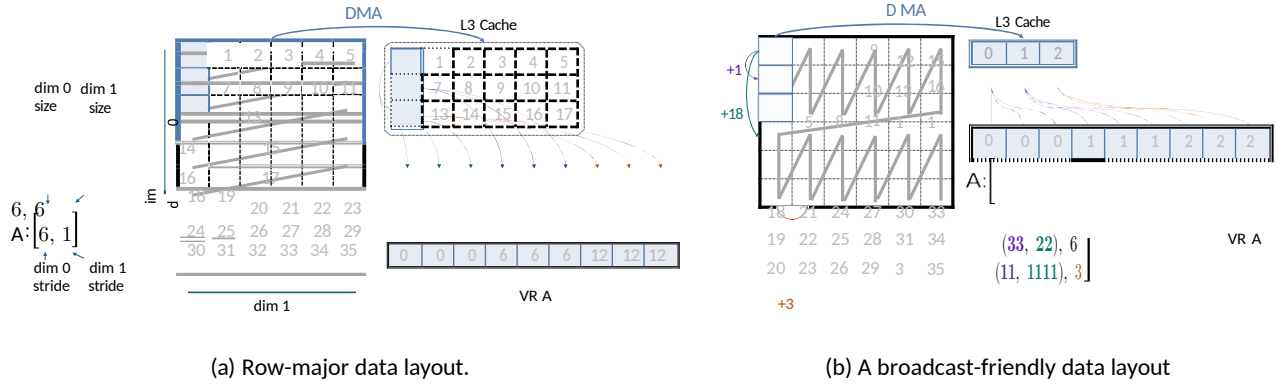


Figure 11: Broadcast-friendly data layout example – (a) a row-major data layout requires a lookup table of size 18. (b) A broadcast-friendly data layout only requires a lookup table size of size 3.

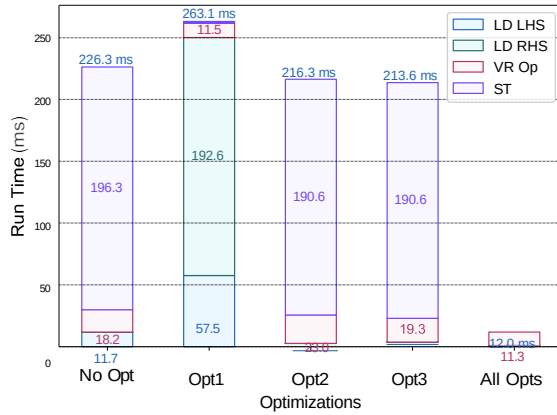


Figure 12: Binary matrix multiplication runtime breakdown with different optimizations. Opt1: communication-aware reduction mapping. Opt2: DMA Coalescing. Opt3: broadcast-friendly data layout.

Table 6: Statistics of the phoenix benchmark suite.

Application	Input Size	#Inst. on CPU	#Inst. of APU μ Code
Histogram	1.5GB	4.8 billion	110.7 million
Linear Regression	512MB	3.8 billion	1.6 million
Matrix Multiply	1,024×1,024	22.6 billion	69.7 million
Kmeans	128k	0.4 billion	0.04 million
Reverse Index	100MB	4.8 billion	11.0 million
String Match	512MB	101.8 billion	0.09 million
Word Count	10MB	0.7 billion	0.17 million

图 13 对比了 APU 实现与 CPU 基线的延迟。相较于单线程 CPU，应用了全部优化的 APU 实现平均加速比为 41.8×（算术平均）、14.4×（几何平均），峰值加速比达到 128.3×。相较于多线程 CPU 执行，APU 的平均加速比为 12.5×（算术平均）、2.6×（几何平均），最大加速比为 68.1×。

5.2.1 Results Analysis. 图 13 中展示的 APU 实现包括未优化基线，以及仅应用通信感知归约映射（Opt1）、仅 DMA 合并（Opt2）、仅广播友好数据布局（Opt3）、以及三种优化同时应用（APU 全部优化）的版本。

Table 7: Phoenix benchmark suite latency measured vs. analytical framework.

Application	Meas. Latency (ms)	Predicted (ms)	Error
Histogram	1644.8	1650.1	+0.32%
Linear Regression	92.3	94.5	+2.3%
Matrix Multiply	421.3	402.5	-4.5%
Kmeans	1.6	1.4	-6.2%
Reverse Index	182.0	181.1	-0.49%
String Match	90.9	92.6	+1.8%
Word Count	3.2	3.1	-3.1%

单独来看，通信感知归约映射在涉及大规模数据比较或距离计算的工作负载中提供了显著收益，例如 k-means、反向索引、字符串匹配和词频统计。DMA 合并需要在需要数据复制的情

况（如矩阵乘法）或可通过打包输入数据提高向量寄存器利用率（如线性回归、字符串匹配）中降低了数据移动开销。广播友好数据布局在通过查找操作广播标量值时表现良好，例如在 k-means 中，尽管这种机会通常需要其他优化先被应用才能显现。总体来看，三种优化同时应用带来的性能提升始终高于任何单独优化的效果。

对基准套件的完全优化结果表明，SRAM 内计算平台最适合特定子集的应用。优化后的 APU 实现在线性回归、k-means、字符串匹配和词频统计等高度数据并行且 VR 内计算开销较小的应用中超过了多线程 CPU。通过提出的优化，大多数算术操作被高效映射到 VR 间的按元素操作指令，同时降低了数据复制开销。相比之下，其他应用（如直方图、矩阵乘法、反向索引）由于算法特性仍频繁涉及 VR 内操作和细粒度元素访问，限制了 SRAM 内计算加速带来的性能提升。

5.2.2 Analytical framework validation. 我们使用 Phoenix 基准套件验证分析框架，通过对比实际测量延迟与预测延迟来评估准确性。表 7 总结了八个基准的结果。平均来看，分析框架的预测准确率为 97.3%，最大误差为 6.2%。

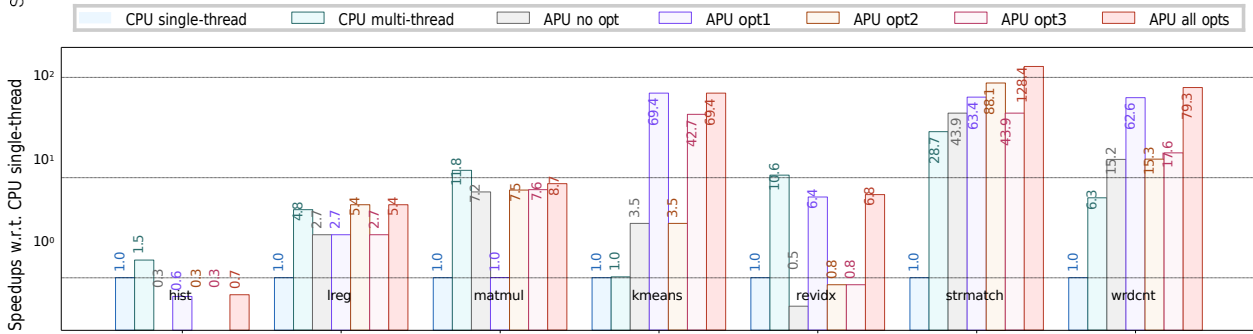


Figure 13: Latency comparison across workloads from the Phoenix Benchmark Suite, normalized to the single-threaded Intel Xeon Gold CPU baseline. Opt1: communication-aware reduction mapping. Opt2: DMA Coalescing. Opt3: broadcast-friendly data layout.

主要误差来源于模型无法考虑内存子系统细节或缓存行为。

5.3 Retrieval-Augmented Generation (RAG)

检索增强生成 (RAG, Retrieval-Augmented Generation) 通过在生成过程中检索相关知识来提升语言模型的响应质量 [16, 34]。其方法是对查询和文档进行嵌入，然后执行向量相似度搜索 [42]。对于大规模语料库，CPU/GPU 通常需要使用近似最近邻搜索 (ANNS)，以精度换取延迟和内存占用。这可能导致显著的精度损失 (Llama-8B 和 Llama-80B 的损失范围为 22%–53% [40])，相较于精确最近邻搜索 (ENNS)。而具备大规模并行能力的 SRAM 内计算平台能够高效加速

ENNS，从而避免这一折衷。本文研究了针对 RAG 的优化 SRAM 内计算 ENNS，重点关注延迟和能耗优势。

5.3.1 Experimental Setup. 我们在 GSI APU 上实现了 ENNS RAG 检索流程。然而，该设备有限的 DDR 带宽 (23.8 GB/s) 会不可避免地形成片外内存瓶颈，影响公平的性能对

比。为缓解这一问题，我们通过模拟 HBM2e 内存 (16 GB, 2 rank, 8 通道, 1.6 GHz, 峰值带宽 380–420 GB/s)，利用 Ramulator 2 [35] 和 DRAMPower 5.0 [12] 来建模更具代表性的片外内存系统。所呈现的 SRAM 内计算性能结果均包含这些模拟的片外内存时序，而其他组件——包括片上数据移动、计算和系统开销——则直接在 GSI APU 硬件上测量。生成模型使用 Llama3.1-8B [20] (16 位数值格式)，问题采样自 Natural Questions (NQ) 数据集 [28]。评估系统包括两块 GPU (1 块专用于生成，另一块用于检索)、CPU 及 GSI

APU。生成在单块 GPU 上运行，检索通过 ENNS 在三种语料库规模 (10GB、50GB、200GB) 上分别在 CPU、GPU 和 SRAM 内计

算加速器上执行。每个语料库被分割为 16,384 个 token 的示，块。结果是，10GB 语料包含 16.3 万块 (嵌入大小 120MB)，50GB 包含 81.9 万块 (嵌入大小 600MB)，200GB 增至

包含 330 万块 (嵌入大小 2.4GB)。

对于 GPU 和 SRAM 内计算加速器，语料嵌入在工作负载开始时一次性传入设备内存，后续所有查询均无需重新加载嵌入。同时，语料块保存在 CPU 主内存中。在以下结果中，我们报告每个平台的交互延迟 (time-to-interactive latency，也称为首 token 延迟 time-to-first-token latency)，这是评估 LLM 推理系统交互性的重要指标。所有延迟结果为 10 次查询的平均值。

5.3.2 Software Configurations. 我们使用 FAISS [14]

在 CPU 和 GPU 上评估 RAG 性能。FAISS 是一种广泛采用的高效密集向量相似度搜索与聚类库。实验使用 FAISS v1.7.2，执行 ENNS 内积搜索 (IndexFlat)，CPU 上利用 AVX512 指令集和基于 OpenMP 的多线程加速。

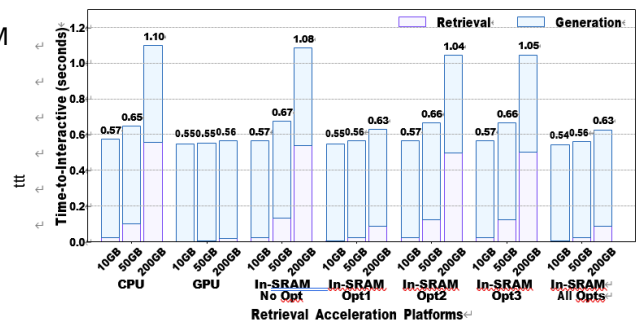


Figure 14: Inference time breakdown of CPU, GPU, vs. compute-in-SRAM with and without optimizations. The Llama3.1-8B generative model runs on a dedicated GPU. Opt1: communication-aware reduction mapping. Opt2: DMA Coalescing. Opt3: broadcast-friendly data layout.

5.3.3 End-to-End RAG Performance. 如图 14 所

随着语料规模增长，检索在端到端推理时间中所占比例逐渐上升 (CPU 检索：10 GB 时为 4.3%，200 GB 时为 50.5%)。我们在 SRAM 内计算加速器上通过通信感知归约映射和广播友好查询布局优化内积搜索，将归约操作映射到 inter-VR 指令。与 CPU 相比，检索加速比在 10/50/200 GB 分别为 7.5×/5.4×/7.3×，带来 1.05×/1.15×/1.78× 的端到端性能提升。相比未经优化的 SRAM 内计算基线，检索延迟最高下降 6.5×。优化后的系统实现了 GPU 级的端到端延迟，凸显了检索侧加速的有效性。

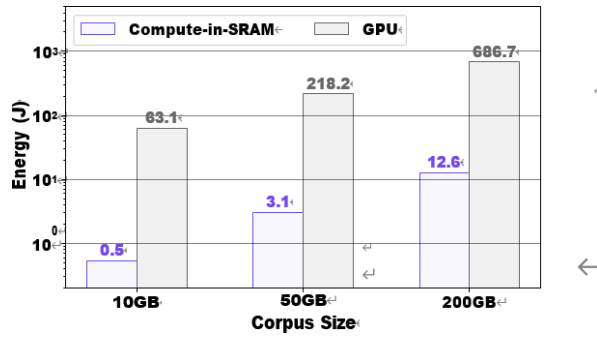


Figure 15: Top-5 retrieval process energy comparison with GPU. Results are measured on GSI Leda-E APU and NVIDIA A6000 GPU.

5.3.4 优化影响与检索延迟细分

在未优化情况下，针对 10/50/200 GB 数据集，RAG 的 APU 检索延迟为 21.8/129.5/539.2ms，与 CPU 性能相当，但比 GPU 慢。通信感知的归约映射（opt1）解决了输出数据移动的瓶颈，将检索延迟降低至 4.0/21.0/86.1ms。DMA 合并（opt2）和广播友好布局（opt3）单独提升有限，但与 opt1 结合时效果显著；三者同时使用时，延迟降至 3.9/20.6/84.2ms。图 14 显示，虽然 opt2 和 opt3 单独效果有限，但它们通过改善数据移动和向量利用率增强了 opt1 的性能。

如表 8 所示，大部分优化后的 SRAM 内计算检索加速来自距离计算阶段。关键在于通信感知归约映射——它将内积归约映射到 VR 间操作以减少 VR 内部的数据移动，并改善对齐（例如，200 GB 时 embedding 加载时间从 8.2ms 降至 6.1ms）。广播友好布局进一步降低了查询广播开销并提高了向量重用率，带来额外的计算时间节省。

5.3.5 与 GPU 的能效比较

我们将优化后的 SRAM 内计算加速器在 Top-5 检索任务中的性能与 NVIDIA A6000 GPU 进行对比，并使用 nvidia-smi 测量 GPU 能耗。如图 15 所示，APU 的能效比 GPU 高 54.4×–117.9×，分别在 10 GB、50 GB 和 200 GB 数据集上节省能耗 99.2%、98.6% 和 98.2%。在 200 GB 数据集上，APU 能耗主要由静态功耗主导

（71.4%），其次是计算（24.7%）、DRAM（2.7%）、其他（1.1%）和缓存（0.005%）；较小数据集的分布类似，表明静态功耗占主导，而计算功耗随工作量增长呈适度上升。

6 相关工作

计算-内存（Compute-in-Memory）架构通过减少内存与计算单元之间的数据移动，为数据密集型应用提供了高能效的潜力。智能内存（IRAM）[36–38] 是最早尝试之一，将计算逻辑直接集成到 DRAM 中，展示了内存与向量处理耦合的潜力。基于此，VIRAM [17] 引入了带嵌入式 DRAM 的完整向量处理器，加速带宽受限的工作负载。DIVA [4] 将 SPMD（单程序多数据）模型引入内存处理（PNM），实现更灵活的并行性，而 FlexRAM [5,10] 在嵌入式 DRAM 系统中扩展了这一

Table 8: Compute-in-SRAM retrieval latency breakdown across corpus size with and without optimizations.

Corpus Size	Compute-in-SRAM No Opt			Compute-in-SRAM All Opts		
	10 GB	50 GB	200 GB	10 GB	50 GB	200 GB
Load Embedding*	0.4 ms	2.0 ms	8.2 ms	0.3 ms	1.5 ms	6.1 ms
Load Query	10 μs	11 μs	10 μs	62 μs	62 μs	65 μs
Calc Distance	21.0 ms	126.5 ms	527.9 ms	3.1 ms	18.0 ms	74.6 ms
Top-K Aggregation	69 μs	325 μs	1.30 ms	72 μs	317 μs	1.24 ms
Return Top-K	14 μs	14 μs	14 μs	15 μs	16 μs	16 μs
Total	21.8 ms	129.5 ms	539.2 ms	3.9 ms	20.6 ms	84.2 ms

* Load embedding latency reflects simulated HBM2e performance; all other values are measured on GSI APU hardware.

模型，强调了通用计算-内存平台可编程抽象的重要性。

SRAM 内计算架构被用于实现布尔计算 [2,3,8,26]、乘法（MAC）[6,9,24,25,29,30,32,45,48] 和关联计算 [15,21,39,43]。Jeloka 等 [26] 引入了位线计算技术，使 SRAM 行之间能够进行按位逻辑操作。Compute Caches [1] 将位线计算应用于将片上多核缓存转化为逻辑计算引擎。SRAM 技术在原位 MAC 操作中也表现出高效性，这得益于 SRAM 位单元的高开/关阻比 [9,27,32,49]。关联计算利用搜索和多写等原语实现内存内计算 [15,43]，在现代技术下重新受到关注。CAPE [11] 展示了基于 CMOS 的关联引擎，具有高可编程性和低面积成本。

APU 微基准测试

此前工作将 RISC-V 向量抽象映射到 APU [19]，加速了基因组学内核 [18]，并实现了密码学原语 [33]，但这些手动优化微内核主要突出架构特性，而非系统级行为。相比之下，我们评估了端到端的 Phoenix 和 RAG 工作负载，提供详细的性能表征和对实际计算及内存需求的分析。

7 Conclusion

本文对 SRAM 内计算设备在真实工作负载下进行了全面评估。我们的分析框架强调了通用 SRAM 内计算的关键优化原则。通过数据移动优化，包括通信感知归约映射、DMA 合并以及广播友好数据布局，我们将 RAG 检索阶段加速了 5.4×–7.5×，并将交互延迟缩短了 1.1×–1.8×。我们的系统在性能上可与 NVIDIA A6000 GPU 匹配，同时能耗降低 54.4×–117.9×，凸显了 SRAM 内计算架构的实用性和高效性。

References

- [1] Shaizeen Aga, Supreet Jeloka, Arun Subramaniyan, Satish Narayanasamy, David Blaauw, and Reetuparna Das. 2017. Compute Caches. In 2017 IEEE International Symposium on High Performance Computer Architecture (HPCA) (Austin, TX, USA). IEEE Computer Society, Los Alamitos, CA, USA, 481–492. <https://doi.org/10.1109/HPCA.2017.21>
- [2] Amogh Agrawal, Akhilesh Jaiswal, Chankyu Lee, and Kaushik Roy. 2018. X-SRAM: Enabling in-memory Boolean computations in CMOS static random access memories. *IEEE Transactions on Circuits and Systems I: Regular Papers* 65, 12 (2018), 4219–4232.
- [3] Amogh Agrawal, Akhilesh Jaiswal, Deboleena Roy, Bing Han, Gopalakrishnan Srinivasan, Aayush Ankit, and Kaushik Roy. 2019. Xcel-RAM: Accelerating binary neural networks in high-throughput SRAM compute arrays. *IEEE Transactions on Circuits and Systems I: Regular Papers* 66, 8 (2019), 3064–3076.
- [4] Junwhan Ahn, Sungpack Hong, Sungjoo Yoo, Onur Mutlu, and Kiyoung Choi. 2015. A Scalable Processing-in-Memory Accelerator for Parallel Graph Processing. In *ISCA '15: Proceedings of the 42nd Annual International Symposium on Computer Architecture* (Portland, OR, USA). Association for Computing Machinery, New York, NY, USA, 105–117. <https://doi.org/10.1145/2749469.2750386>
- [5] Junwhan Ahn, Sungjoo Yoo, Onur Mutlu, and Kiyoung Choi. 2015. PIM-enabled instructions: A low-overhead, locality-aware processing-in-memory architecture. *ACM SIGARCH Computer Architecture News* 43, 35 (2015), 336–348.
- [6] Khalid Al-Hawaj, Olalekan Afuye, Shady Agwa, Alyssa Apsel, and Christopher Batten. 2020. Towards a Reconfigurable Bit-Serial/Bit-Parallel Vector Accelerator Using In-Situ Processing-in-SRAM. In 2020 IEEE International Symposium on Circuits and Systems (ISCAS) (Seville, Spain (Virtual)). Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 1–5. <https://doi.org/10.1109/ISCAS45731.2020.9181068>
- [7] Khalid Al-Hawaj, Tuan Ta, Nick Cebry, Shady Agwa, Olalekan Afuye, Eric Hall, Courtney Golden, Alyssa B. Apsel, and Christopher Batten. 2023. EVE: Ephemeral Vector Engines. In 2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA) (Montreal, QC, Canada). IEEE Computer Society, Los Alamitos, CA, USA, 691–704. <https://doi.org/10.1109/HPCA56546.2023.10071074>
- [8] Daniel Bankman, Lita Yang, Bert Moons, Marian Verhelst, and Boris Murmann. 2018. An Always-On 3.8 μ mJ 86% CIFAR-10 mixed-signal binary CNN processor with all memory on chip in 28-nm CMOS. *IEEE Journal of Solid-State Circuits* 54, 1 (2018), 158–172.
- [9] Avishke Biswas and Anantha P. Chandrakasan. 2018. Conv-RAM: An Energy-Efficient SRAM with Embedded Convolution Computation for Low-Power CNN-Based Machine Learning Applications. In 2018 IEEE International Solid-State Circuits Conference (ISSCC) (San Francisco, CA, USA). Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 488–490. <https://doi.org/10.1109/ISSCC.2018.8310397>
- [10] Jay B. Brockman, Shyamkumar Thoziyoor, Shannon K. Kuntz, and Peter M. Kogge. 2004. A Low Cost, Multithreaded Processing-in-Memory System. In *WMPPI '04: Proceedings of the 3rd Workshop on Memory Performance Issues, in conjunction with the 31st International Symposium on Computer Architecture* (Munich, Germany). Association for Computing Machinery, New York, NY, USA, 16–22. <https://doi.org/10.1145/1054943.1054946>
- [11] Helena Caminal, Kailin Yang, Srivatsa Srinivasa, Akshay Krishna Ramanathan, Khalid Al-Hawaj, Tianshu Wu, Vijaykrishnan Narayanan, Christopher Batten, and José F. Martínez. 2021. CAPE: A Content-Addressable Processing Engine. In 2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA) (Seoul, South Korea (Virtual)). IEEE Computer Society, Los Alamitos, CA, USA, 557–569. <https://doi.org/10.1109/HPCA51647.2021.00054>
- [12] Karthik Chandrasekar, Christian Weis, Yonghui Li, Sven Goossens, Matthias Jung, Omar Naji, Benny Akesson, Norbert Wehn, and Kees Goossens. 2024. DRAM- Power: Open-source DRAM Power & Energy Estimation Tool. TU Kaiserslautern, Microelectronic Systems Design (MSD) Research Group. <http://www.drampower.info>
- [13] Johannes de Fine Licht, Maciej Besta, Simon Meierhans, and Torsten Hoefer. 2020. Transformations of high-level synthesis codes for high-performance computing. *IEEE Transactions on Parallel and Distributed Systems* 32, 5 (2020), 1014–1029.
- [14] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss Library. <https://doi.org/10.48550/arXiv.2401.08281> [cs.LG]
- [15] Caxton C. Foster. 1976. Content Addressable Parallel Processors. Van Nostrand Reinhold Company, New York, NY, USA.
- [16] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. <https://doi.org/10.48550/arXiv.2312.10997> [cs.CL]
- [17] Joseph Gebis, Sam Williams, David Patterson, and Christos Kozyrakis. 2004. VIRAM1: A Media-Oriented Vector Processor with Embedded DRAM. In *DAC '04: Proceedings of the 41st Annual Design Automation Conference – Student Design Contest* (San Diego, CA, USA). Association for Computing Machinery, New York, NY, USA, 6 pages. <https://cs.stanford.edu/~christos/publications/2004.dac.iram.pdf> Student Design Contest paper.
- [18] Courtney Golden, Dan Ilan, Nicholas Cebry, and Christopher Batten. 2023. Accelerating Seed Location Filtering in DNA Read Mapping Using a Commercial Compute-in-SRAM Architecture. In *Proceedings of the 5th Workshop on Accelerator Architecture in Computational Biology and Bioinformatics (AACBB)* (Orlando, FL, USA). Association for Computing Machinery, New York, NY, USA, 10 pages. <https://www.csl.cornell.edu/~cbatten/pdfs/golden-apu-filtering-aacbb2023.pdf> Workshop held in conjunction with ISCA'50 (FCRC 2023); see also arXiv:2401.11685.
- [19] Courtney Golden, Dan Ilan, Caroline Huang, Niansong Zhang, Zhiru Zhang, and Christopher Batten. 2024. Supporting a Virtual Vector Instruction Set on a Commercial Compute-in-SRAM Accelerator. *IEEE Computer Architecture Letters* 23, 1 (Jan 2024), 29–32. <https://doi.org/10.1109/LCA.2023.3341389>
- [20] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The Llama 3 Herd of Models. <https://doi.org/10.48550/arXiv.2407.21783> arXiv:2407.21783 [cs.LG]
- [21] Qing Guo, Xiaochen Guo, Ravi Patel, Engin Ipek, and Eby G. Friedman. 2013. AC-DIMM: Associative Computing with STT-MRAM. In *ISCA '13: Proceedings of the 40th Annual International Symposium on Computer Architecture* (Tel-Aviv, Israel). Association for Computing Machinery, New York, NY, USA, 189–200. <https://doi.org/10.1145/2485922.2485939>
- [22] Linley Gwennap. 2020. In-Memory Acceleration for Big Data. Technical Report. The Linley Group. <https://gsitechnology.com/wp-content/uploads/2023/01/GSIT-Gemini-WP-Final-Linley.pdf>
- [23] Bastian Hagedorn, Bin Fan, Hanfeng Chen, Cris Cecka, Michael Garland, and Vinod Grover. 2023. Graphene: An IR for Optimized Tensor Computations on GPUs. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS '23)* (Vancouver, BC, Canada). Association for Computing Machinery, New York, NY, USA, 302–313. <https://doi.org/10.1145/3582016.3582018>
- [24] Mohsen Imani, Saransh Gupta, Yeseong Kim, and Tajana Rosing. 2019. FloatPIM: In-Memory Acceleration of Deep Neural Network Training with High Precision. In *ISCA '19: Proceedings of the 46th International Symposium on Computer Architecture* (Phoenix, AZ, USA). Association for Computing Machinery, New York, NY, USA, 802–815. <https://doi.org/10.1145/3307650.3322237>
- [25] M. Ishida, T. Kawakami, A. Tsuji, N. Kawamoto, M. Motoyoshi, and N. Ouchi. 1998. A Novel 6T-SRAM Cell Technology Designed with Rectangular Patterns Scalable Beyond 0.18 μ m Generation and Desirable for Ultra High Speed Operation. In 1998 IEEE International Electron Devices Meeting (IEDM) Technical Digest (San Francisco, CA, USA). Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 201–204. <https://doi.org/10.1109/IEDM.1998.746322>
- [26] Supreet Jeloka, Naveen Bharathwaj Akes, Dennis Sylvester, and David Blaauw. 2016. A 28 nm configurable memory (TCAM/BCAM/SRAM) using push-rule 6T bit cell enabling logic-in-memory. *IEEE Journal of Solid-State Circuits* 51, 4 (2016), 1009–1021.
- [27] Zhewei Jiang, Shihui Yin, Jae-Sun Seo, and Mingoo Seok. 2020. C3SRAM: An in-memory-computing SRAM macro based on robust capacitive coupling computing mechanism. *IEEE Journal of Solid-State Circuits* 55, 7 (2020), 1888–1897.
- [28] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. 2019. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics* 7 (2019), 453–466.
- [29] Ann Franchesca Laguna, Arman Kazemi, Michael T. Niemier, and X. Sharon Hu. 2021. In-Memory Computing Based Accelerator for Transformer Networks for Long Sequences. In *Proceedings of the 2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)* (Grenoble, France (Virtual)). Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 1839–1844. <https://doi.org/10.23919/DATE51398.2021.9474146>
- [30] Ann Franchesca Laguna, Mohammed Mehdi Sharifi, Arman Kazemi, Xunzhao Yin, Michael Niemier, and X. Sharon Hu. 2022. Hardware-Software Co-Design of an In-Memory Transformer Network Accelerator. *Frontiers in Electronics* 3, Article 847069 (Apr 2022), 21 pages. <https://doi.org/10.3389/felec.2022.847069>
- [31] Phuoc-Hoan Charles Le and Xinlin Li. 2023. BinaryViT: Pushing Binary Vision Transformers Towards Convolutional Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)* (Vancouver, BC, Canada). IEEE Computer Society, Los Alamitos, CA, USA, 4664–4673. <https://doi.org/10.1109/CVPRW59228.2023.00492>
- [32] Eunyoung Lee, Taeyoung Han, Donguk Seo, Gicheol Shin, Jaerok Kim, Seonho Kim, Soyoun Jeong, Johnny Rhe, Jaehyun Park, Jong Hwan Ko, et al. 2021. A charge-domain scalable-weight in-memory computing macro with dual-SRAM architecture for precision-scalable DNN accelerators. *IEEE Transactions on Circuits and Systems I: Regular Papers* 68, 8 (2021), 3305–3316.
- [33] Kaitlyn Lee, Brian Donnelly, Tomer Sery, Dan Ilan, Bertrand Cambou, and Michael Gowanlock. 2023. Evaluating Accelerators for a High-Throughput Hash-Based Security Protocol. In *Proceedings of the 52nd International Conference on Parallel Processing Workshops (ICPP-W '23)* (Salt Lake City, UT, USA). Association for Computing Machinery, New York, NY, USA, 40–49. <https://doi.org/10.1145/>

- [34] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.
- [35] Haocong Luo, Yahya Can Tu, F. Nisa Bostanci, Ataberk Olgun, A. Giray Yaglikci, and Onur Mutlu. 2024. Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator. *IEEE Computer Architecture Letters* 23, 1 (2024), 112–116. <https://doi.org/10.1109/LCA.2023.3333759>
- [36] Mark Oskin, Frederic T. Chong, and Timothy Sherwood. 1998. Active Pages: A Computation Model for Intelligent Memory. In *Proceedings of the 25th Annual International Symposium on Computer Architecture (ISCA '98)* (Barcelona, Spain). IEEE Computer Society, Los Alamitos, CA, USA, 192–203. <https://doi.org/10.1109/ISCA.1998.694774>
- [37] David Patterson, Thomas Anderson, Neal Cardwell, Richard Fromm, Kimberly Keeton, Christoforos Kozyrakis, Randi Thomas, and Katherine Yelick. 1997. A case for intelligent RAM. *IEEE micro* 17, 2 (1997), 34–44.
- [38] David Patterson, Thomas Anderson, Neal Cardwell, Richard Fromm, Kimber-ley Keeton, Christoforos Kozyrakis, Randi Thomas, and Katherine Yelick. 1997. Intelligent RAM (IRAM): Chips that Remember and Compute. In *1997 IEEE International Solid-State Circuits Conference (ISSCC) Digest of Technical Papers* (San Francisco, CA, USA). Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 224–225. <https://doi.org/10.1109/ISSCC.1997.585348>
- [39] Jerry Potter, Johnnie Baker, Stephen Scott, Arvind Bansal, Chokchai Leangsuksun, and Chandra Asthagiri. 1994. ASC: an associative- computing paradigm. *Computer* 27, 11 (1994), 19–25.
- [40] Derrick Quinn, Mohammad Nouri, Neel Patel, John Salihu, Alireza Salemi, Su Khan Lee, Hamed Zamani, and Mohammad Alian. 2025. Accelerating Retrieval-Augmented Generation. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (Rotterdam, Netherlands) (ASPLOS '25). Association for Computing Machinery, New York, NY, USA, 15–32. <https://doi.org/10.1145/3669940.3707264>
- [41] Colby Ranger, Ramanan Raghuraman, Arun Penmetsa, Gary Bradschi, and Chris-tos Kozyrakis. 2007. Evaluating MapReduce for Multi-core and Multiprocessor Systems. In *2007 IEEE 13th International Symposium on High Performance Com-puter Architecture (HPCA)* (Phoenix, AZ, USA). IEEE Computer Society, Los Alamitos, CA, USA, 13–24. <https://doi.org/10.1109/HPCA.2007.346181>
- [42] Alireza Salemi and Hamed Zamani. 2024. Evaluating Retrieval Quality in Retrieval-Augmented Generation. In *SIGIR '24: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Washington, DC, USA). Association for Computing Machinery, New York, NY, USA, 2395–2400. <https://doi.org/10.1145/3626772.3657957>
- [43] Gordon E. Sayre. 1976. Staran: An Associative Approach to Multiprocessor Architecture. In *Computer Architecture: Workshop of the Gesellschaft für In-formatik, Erlangen, May 22–23, 1975*. Springer, Berlin, Heidelberg, 199–221. https://doi.org/10.1007/978-3-642-66400-7_9
- [44] GSI Technology. 2023. GSI Technology's Gemini-I® APU Showcased in "In-Memory Acceleration for Big Data". <https://ir.gsitechnology.com/news-releases/news-release-details/gsi-technologys-gemini-ir-apu-showcased-memory-acceleration-big>.
- [45] Fengbin Tu, Zihan Wu, Yiqi Wang, Ling Liang, Liu Liu, Yufei Ding, Leibo Liu, Shaojun Wei, Yuan Xie, and Shouyi Yin. 2023. TranCIM: Full-Digital Bitline-Transpose CIM-based Sparse Transformer Accelerator With Pipeline/Parallel Reconfigurable Modes. *IEEE Journal of Solid-State Circuits* 58, 6 (Jun 2023), 1798–1809. <https://doi.org/10.1109/JSSC.2022.3213542>
- [46] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. 2023. BitNet: Scaling 1-Bit Transformers for Large Language Models. <https://doi.org/10.48550/arXiv.2310.11453> arXiv:2310.11453 [cs.CL]
- [47] Yue Zha and Jing Li. 2020. Hyper-AP: Enhancing Associative Processing Through a Full-Stack Optimization. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)* (Valencia, Spain (Virtual)). Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 846–859. <https://doi.org/10.1109/ISCA45697.2020.00074>
- [48] Bo Zhang, Shihui Yin, Minkyu Kim, Jyotishman Saikia, Soonwan Kwon, Sung-meon Myung, Hyunsoo Kim, Sang Joon Kim, Jae-Sun Seo, and Mingoo Seok. 2023. PIMCA: A Programmable In-Memory Computing Accelerator for Energy-Efficient DNN Inference. *IEEE Journal of Solid-State Circuits* 58, 5 (May 2023), 1436–1449. <https://doi.org/10.1109/JSSC.2022.3211290>
- [49] Jintao Zhang, Zhuo Wang, and Naveen Verma. 2017. In-memory computation of a machine-learning classifier in a standard 6T SRAM array. *IEEE Journal of Solid-State Circuits* 52, 4 (2017), 915–924.
- [50] Yichi Zhang, Junhao Pan, Xinheng Liu, Hongzheng Chen, Deming Chen, and Zhiru Zhang. 2021. FracBNN: Accurate and FPGA-Efficient Binary Neural Networks with Fractional Activations. In *FPGA '21: Proceedings of the 2021 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (Virtual Event, USA)*. Association for Computing Machinery, New York, NY, USA, 171–182. <https://doi.org/10.1145/3431920.3439296>
- [51] Yichi Zhang, Zhiru Zhang, and Lukasz Lew. 2022. PokeBNN: A Binary Pursuit of Lightweight Accuracy. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (New Orleans, LA, USA). IEEE Computer Society, Los Alamitos, CA, USA, 12475–12485. <https://doi.org/10.1109/CVPR52688.2022.01215>